Koneru Lakshmaiah Education Foundation
(Category -1, Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
❖ Approved by AICTE ❖ ISO 21001:2018 Certified
Campus: Green Fields, Vaddeswaram - 522 302, Guntur District, Andhra Pradesh, INDIA.
Phone No. +91 8645 - 350 200; www.klef.ac.in; www.klef.edu.in; www.kluniversity.in
Admin Off: 29-36-38, Museum Road, Governorpet, Vijayawada - 520 002. Ph: +91 - 866 - 3500122, 2576129

SMART FILE ORGANIZER

A Python-Based File Classification, Duplicate Detection and Recovery System

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of

MASTER OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

by

Mercypriya Talari

Roll No: 2601050114

under the supervision of

Dr. Sai Durga Tejaswi Vinnakota

Profesor

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation (KLEF)

Vaddeswaram, Guntur District, Andhra Pradesh, India

October 2026

DECLARATION

I hereby declare that the project report entitled “Smart File Organizer” is prepared for academic purposes. The report describes the design and implementation of a Python-based utility for file organization, duplicate detection, metadata inspection, monitoring and recovery. Any test results included in the final submitted copy should be taken from actual executions of the project. Sources used in preparing the report are acknowledged in the references.

Place:Vaddeswaram
Date:13-10-2026_

Signature of the Student: Mercypriya Talari(2601050114)

ACKNOWLEDGEMENT

I would like to express my gratitude to my project guide and faculty members for their guidance and support. I also acknowledge the Python, Streamlit, Ubuntu and Microsoft WSL documentation, which provide useful references for implementing and understanding the technologies used in this project. Finally, I thank my institution for providing an opportunity to develop this practical software project.

ABSTRACT

Managing a busy Downloads or working directory can become difficult when files of different types are mixed together. Finding documents, images, videos, audio files and archives manually takes time and may lead to mistakes. The Smart File Organizer project is a Python-based application designed to classify files into folders based on their extensions. The supplied implementation includes a Streamlit dashboard, a command-line organizer, duplicate detection using SHA-256, metadata inspection, file monitoring, concurrent organization, movement history and an undo operation.

The organizer groups common formats into Documents, Images, Videos, Audio and Archives, while unrecognized extensions are placed in Others. It creates category folders when required and adds a numeric suffix to a destination filename when a name already exists, helping avoid overwriting. The duplicate finder calculates SHA-256 hashes in chunks and reports files with matching content hashes. The metadata module reports file name, size, extension and modification time. The project also includes a four-worker ThreadPoolExecutor implementation and a benchmark script intended to compare sequential and concurrent processing.

The project is intended as a learning and productivity tool for Linux/Ubuntu and WSL2 environments. The dashboard is implemented with Streamlit, while file operations use Python standard-library modules. Since organizing files changes their locations, testing should first be performed on a folder containing disposable sample files. Performance values should be recorded only after running the benchmark in the target environment.

Keywords: file organization, Python, Streamlit, SHA-256, metadata, concurrency, Linux, recovery.

TABLE OF CONTENTS

Section	Page
Abstract	4
1. Introduction and Problem Definition	6
2. Literature Review and Theoretical Foundations	7
3. Proposed Methodology and System Architecture	8
4. Implementation, Testing and Discussion	10
5. Conclusion, Limitations and Future Scope	12
References	14

Note: Page numbers are indicative. Update the table of contents after editing and final pagination.

1. INTRODUCTION AND PROBLEM DEFINITION

1.1 Introduction

Files are created and downloaded in many formats. When a folder contains hundreds of mixed files, manual sorting becomes repetitive and makes it harder to locate information. A file organizer automates this task by checking file extensions and moving files into suitable category folders. This project adds a dashboard and supporting utilities to make organization, inspection and recovery easier to demonstrate.

1.2 Problem statement

The problem addressed is the time and effort required to manage mixed files in a directory. Manual sorting is slow, duplicate copies can consume storage, and users may need to inspect file properties or reverse a file movement. The project therefore aims to automate file classification, identify duplicate content, display basic metadata, monitor new files and retain movement history for an undo operation.

1.3 Objectives

The main objectives are: (1) organize files according to their extensions; (2) create category directories automatically; (3) prevent filename collisions during moves; (4) identify potential duplicate files using SHA-256; (5) display file metadata; (6) record file movements and support undoing the most recent movement; (7) provide a concurrent organization implementation; (8) monitor a directory for new files; and (9) offer a Streamlit dashboard to launch project operations.

1.4 Scope

The supplied code is designed for local directories in a Linux/Ubuntu or WSL2 environment. It supports a defined set of common extensions and places other extensions in an Others category. The application is suitable for demonstrations and small personal folders. It does not provide cloud synchronization, enterprise access control, a database-backed audit service or a guarantee that every file format is correctly classified. The dashboard and recovery module use configured paths that should be checked and adjusted for the user's machine before execution.

1.5 Significance

Automated organization reduces repetitive manual work and makes a directory easier to navigate. Hash-based duplicate checking compares file content rather than relying only on filenames. Metadata inspection supports quick review of file properties, while movement history provides a limited way to reverse accidental organization. Together, these features demonstrate practical use of filesystem programming, hashing, logging, concurrency and a lightweight Python web interface.

2. LITERATURE REVIEW AND THEORETICAL FOUNDATIONS

2.1 Rule-based file classification

A rule-based organizer maps file extensions to categories. For example, .pdf, .docx and .txt are classified as Documents; .jpg and .png as Images; .mp4 and .mkv as Videos; .mp3 and .wav as Audio; and .zip or .7z as Archives. Extension-based classification is easy to understand and efficient, but it does not inspect the internal contents of a file. A misleading or missing extension may therefore lead to an incorrect category.

2.2 Duplicate detection with SHA-256

A cryptographic hash function converts file content into a fixed-length digest. The project's duplicate finder uses SHA-256 and reads a file in 4096-byte chunks, avoiding the need to load the entire file into memory at once. Files with the same hash are reported as duplicates. Hash comparison is generally more reliable than comparing names or sizes alone, although a production-grade tool may also compare file sizes and confirm byte equality before taking destructive action. The supplied finder reports duplicates; it does not delete them.

2.3 File metadata

Filesystem metadata provides information such as file size, modification timestamp and extension. The metadata module recursively walks a selected folder and prints these properties for each file. This information can help users inspect a directory before organizing it. Metadata availability and timestamp interpretation can vary by operating system and filesystem.

2.4 Logging and recovery

Logging creates a record of file operations and errors, which is useful for debugging and auditing. In this project, organizer.py writes activity to a configured log file, and recovery.py records source and destination paths in a JSON movement-history file. The undo function attempts to move the most recently organized file back to its original path, provided the moved file still exists and the original path is not occupied. This is a basic recovery feature rather than a complete backup system.

2.5 Concurrent file processing

Sequential processing handles one file at a time. The concurrent implementation uses Python's ThreadPoolExecutor with four workers to schedule file moves. Because filesystem work often involves input/output, threads may improve throughput in some environments. Actual improvement depends on file count, storage speed, filesystem behavior and workload. Concurrency can also introduce destination-name races if multiple workers attempt to create the same destination simultaneously, so results should be tested carefully.

2.6 Related tools and project position

Operating systems provide file managers and command-line utilities that can move and inspect files. General-purpose scripting can automate these actions but may require users to remember commands. The Smart File Organizer combines rule-based organization and utility scripts with a Streamlit dashboard. Its main value is educational integration of common filesystem tasks in one small application; it should not be treated as a replacement for backups or a production data-management platform.

3. PROPOSED METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 System overview

The application is composed of independent Python modules and a Streamlit user interface. The dashboard launches the modules as subprocesses and presents controls for core operations. The command-line organizer can also run independently by asking for a folder path. The project uses pathlib for path handling, shutil for moving files, hashlib for hashing, logging for operation records, json for movement history, and concurrent.futures for thread-based processing.

3.2 Architecture diagram

```
User
  |
  v
Streamlit Dashboard (app.py)  <---->  User-selected target folder
  |
  +--> organizer.py              --> Category folders and activity log
  +--> duplicate_finder.py       --> SHA-256 duplicate report
  +--> metadata.py               --> File size/type/modified time
  +--> file_monitor.py           --> Polling monitor and automatic moves
  +--> organizer_concurrent.py   --> Four-worker concurrent moves
  +--> benchmark.py              --> Sequential/concurrent timing
  +--> recovery.py               --> JSON movement history / undo
  +--> run_organizer.sh          --> Shell-based execution
```

3.3 File organization algorithm

Step 1: Read the selected folder path. Step 2: Check whether the folder exists. Step 3: Enumerate the files directly inside the folder. Step 4: Read each file's lowercase extension. Step 5: map the extension to a category, using Others when no rule matches. Step 6: create the category folder if necessary. Step 7: select a destination filename; if it already exists, append a numeric suffix. Step 8: move the file, record its movement and log success; if an exception occurs, log and display the error. Step 9: report that organization has completed.

3.4 Duplicate finder algorithm

Step 1: Traverse the selected directory recursively. Step 2: Skip entries that are not files. Step 3: Read each file in fixed-size chunks and update a SHA-256 digest. Step 4: If the digest has been seen before, report the previous path and the current path. Step 5: Otherwise, store the digest and path for later comparison. The algorithm identifies duplicate content candidates but does not remove files.

3.5 Recovery design

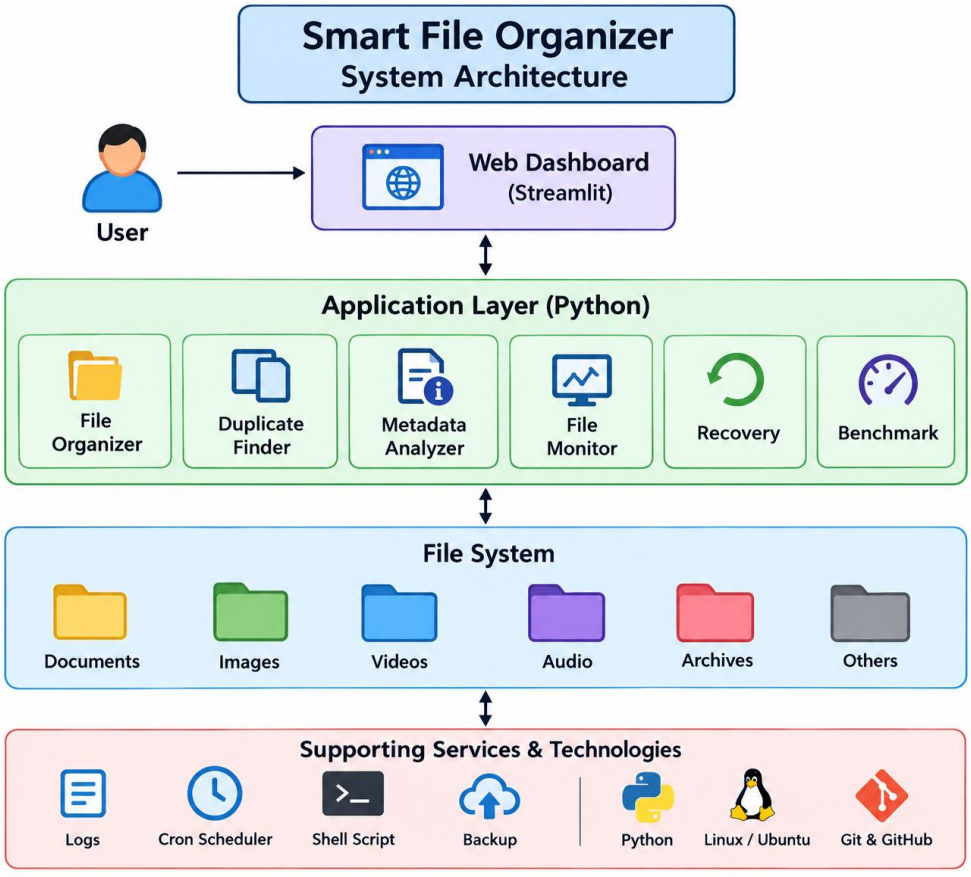
After a file is moved by organizer.py, record_move stores the original source path and new destination path in logs/move_history.json. The undo function loads the history and examines the latest entry. If the destination file is missing, or the source location already contains a file, the operation is not performed. Otherwise, it recreates the source directory if needed, moves the file back and removes the history entry. Users should retain backups because the history file alone cannot recover a deleted or damaged file.

3.6 Main modules

app.py: Streamlit dashboard and controls. organizer.py: sequential file categorization, moves, logging and recovery recording. organizer_concurrent.py: threaded organization using four workers. duplicate_finder.py: SHA-256-based duplicate reporting. metadata.py: recursive metadata display. file_monitor.py: periodic polling for new files and movement into categories. recovery.py: JSON movement history and undo. benchmark.py: benchmark utility. run_organizer.sh: shell script that invokes the organizer for a configured demo folder.

3.7 Technologies used

Programming language: Python 3. User interface: Streamlit. Operating environment: Linux/Ubuntu or WSL2. Filesystem handling: pathlib and shutil. Hashing: hashlib SHA-256. Logs and history: logging and JSON. Concurrency: ThreadPoolExecutor. Automation: Bash and optionally cron. Version control: Git and GitHub.



Proposed Methodology and System Architecture

4. IMPLEMENTATION, TESTING AND DISCUSSION

4.1 Environment and setup

The project README specifies Python 3 and Linux/Ubuntu or WSL2. The dashboard imports Streamlit, so Streamlit must be installed in the active Python environment. The supplied scripts contain configured paths under /mnt/d/smart-file-organizer and /mnt/d/organizer-demo. These paths are appropriate only when the Windows D: drive is mounted in WSL and the project is located at the expected path. Before running the dashboard, update BASE_DIR and other hard-coded paths if the project is stored elsewhere.

4.2 Running the application

Open the project folder in VS Code using a WSL/Ubuntu terminal. Confirm Python with `python3 --version`. Create and activate a virtual environment using `python3 -m venv .venv` and `source .venv/bin/activate`. Install the dashboard dependency with `python -m pip install --upgrade pip` and `pip install streamlit`. Create the logs directory, ensure app.py is the dashboard filename, and start the interface with `streamlit run app.py`. Open the local address printed by Streamlit, usually http://localhost:8501. Test with disposable sample files first because the organizer moves files.

4.3 Test plan

The following tests should be run and their actual outputs recorded before final submission: (1) organize a folder containing .txt, .pdf, .jpg and .mp3 files; (2) verify unsupported extensions go to Others; (3) create two files with the same name in the destination category and verify that a suffix is added; (4) place identical-content files in a test folder and check whether duplicate_finder.py reports both paths; (5) run metadata.py and verify name, size, type and modified timestamp; (6) move a file and run recovery.py to undo the most recent move; (7) start and stop file_monitor.py and observe a new sample file; and (8) run benchmark.py and record its printed timing results. Results must be taken from the user's actual run rather than assumed.

4.4 Test result recording table

Use the table below as a report template. Fill the Result column after running the tests on your system.

Test case	Expected observation	Actual result
File type organization	Files moved into correct category folders	Fill after test
Unsupported extension	File moved into Others	Fill after test
Filename collision	Numeric suffix used; existing file retained	Fill after test
Duplicate finder	Matching SHA-256 files reported	Fill after test
Metadata inspection	Name, size, type and timestamp displayed	Fill after test
Undo last move	Latest recorded move restored if safe	Fill after test
Directory monitor	New top-level file detected and moved	Fill after test
Benchmark	Execution timing printed by benchmark script	Fill after test

4.5 Discussion

The code implements the main organization and inspection operations through small, understandable modules. Filename suffixing helps avoid overwriting files during sequential organization. SHA-256 hashing detects matching content even when file names differ. The recovery history provides a limited undo facility for recorded moves. The dashboard offers a single interface to launch these functions. However, the application relies on fixed filesystem paths, and the concurrent organizer should be evaluated for destination conflicts. The current benchmark notes describe a comparison method and workload sizes, but the available benchmark-results file does not contain actual timing measurements; therefore no numerical performance claim is made in this report.

4.6 Safety and responsible use

Run tests on a temporary folder containing copied sample files. Do not point the organizer at the Windows root, system folders, or a directory containing important files until the behavior has been verified. Back up valuable data before organizing it. Confirm the configured paths before launching dashboard actions. The duplicate finder only reports duplicates in the supplied code, but the organizer itself moves files and changes their locations.

5. CONCLUSION, LIMITATIONS AND FUTURE SCOPE

5.1 Conclusion

Smart File Organizer demonstrates how Python can automate routine filesystem management. It categorizes files using extensions, creates category folders, avoids basic filename collisions, reports duplicate content through SHA-256, displays metadata, monitors a directory, records file movement and supports undoing the latest recorded move. A Streamlit dashboard brings the main operations together, while a concurrent implementation and benchmark script provide a basis for studying processing performance. The project is suitable for an academic demonstration of Python filesystem APIs, hashing, logging, JSON, concurrency and Linux automation.

5.2 Limitations

The current implementation depends on fixed Linux/WSL paths that must match the local setup. Classification depends on file extensions and does not inspect file contents. The duplicate finder reports matches but does not perform a full safe-delete workflow. Recovery works only when the movement history is valid and the destination file remains available. The monitor polls the directory at a fixed interval and only considers files directly inside the watched folder. Concurrent moves can require additional safeguards around collisions. The benchmark results need to be generated on the actual test machine before making performance comparisons.

5.3 Future scope

Possible improvements include a settings page for choosing folders instead of hard-coded paths; a preview or dry-run mode; stronger collision handling in concurrent mode; progress bars and downloadable reports; optional file-content inspection; a safer duplicate review and quarantine workflow; scheduled operation through a managed cron setup; configurable categories and extensions; automated unit tests; and repeatable benchmarks across several file counts and storage environments.

APPENDIX A: COMMANDS TO RUN THE PROJECT

Run these commands in an Ubuntu/WSL terminal after placing the project in /mnt/d/smart-file-organizer and checking that the configured paths match your machine.

```
cd /mnt/d/smart-file-organizer

python3 --version

python3 -m venv .venv

source .venv/bin/activate

python -m pip install --upgrade pip

pip install streamlit

mkdir -p logs

mkdir -p /mnt/d/organizer-demo

streamlit run app.py
```

Before testing, create a disposable sample folder and copy sample files into it. The organizer moves files, so do not initially run it on important folders.

APPENDIX B: PSEUDOCODE

```
PROCEDURE ORGANIZE_FOLDER(folder):
    IF folder does not exist:
        display error
        RETURN
    FOR each regular file directly inside folder:
        extension = lowercase file extension
        category = matching category, otherwise Others
        create category folder if needed
        destination = category folder / original filename
        WHILE destination already exists:
            destination = filename with numeric suffix
        TRY:
            move file to destination
            record source and destination in history
            log success
        CATCH error:
            log and display error
    display completion message
```

REFERENCES

[1] Python Software Foundation, “pathlib — Object-oriented filesystem paths,” Python Documentation. https://docs.python.org/3/library/pathlib.html

[2] Python Software Foundation, “shutil — High-level file operations,” Python Documentation. https://docs.python.org/3/library/shutil.html

[3] Python Software Foundation, “hashlib — Secure hashes and message digests,” Python Documentation. https://docs.python.org/3/library/hashlib.html

[4] Python Software Foundation, “logging — Logging facility for Python,” Python Documentation. https://docs.python.org/3/library/logging.html

[5] Python Software Foundation, “concurrent.futures — Launching parallel tasks,” Python Documentation. https://docs.python.org/3/library/concurrent.futures.html

[6] Streamlit, “Streamlit Documentation.” https://docs.streamlit.io/

[7] Canonical Ltd., “Ubuntu Documentation.” https://help.ubuntu.com/

[8] Microsoft, “Windows Subsystem for Linux Documentation.” https://learn.microsoft.com/windows/wsl/